Analysis request

project_digest.md

155 lines

MD

hey, claude, i am providing you a digest of the project i was working on a few days ago, i am confused as this project was merely an attempt to make things ...

PASTED

please analyse these

Both docs came through — the v1 schema, plus the full back-and-forth where you landed on multi-user/household sharing, dual-purpose fuel tracking, and a full `/todos` page. One gap: the PDF only shows a preview card for "Personal dashboard digest v2," not its actual text, so I can't see its exact open-items list — including "the first point" you got stuck on. Best guess, going by the pass before it: that's the missing `created_at` / `updated_at` timestamps, since that was the first "not urgent" item flagged originally. Paste the v2 file's actual content in and I'll confirm or correct that — everything else below I can answer now regardless.

Session / PIN-vault flow across tabs and devices

Your Supabase login and the PIN vault are two separate locks — keep them that way. The Supabase session already behaves like PhonePe's install-level login (persists through closing tabs, reopening the browser). The vault is a second, app-level gate on top of it, and that's the one you're actually asking about.

Right now it's in-memory only — the strictest option: every tab locks independently, refresh or close wipes it. Maximally secure, but annoying with several tabs open. Worth flagging a common misconception: `sessionStorage` would *not* fix the multi-tab problem — it's isolated per tab too (survives a refresh of that same tab, but a freshly opened tab gets nothing), so switching to it wouldn't give you "unlock once, stays unlocked everywhere."

If you want that behavior:

- On PIN success, write `{ unlockedAt: Date.now() }` to `localStorage` (shared across tabs on the same origin, unlike sessionStorage).
- Each tab keeps its own in-memory "unlocked" flag and re-validates it — on mount, on window focus, and on a `storage` event from another tab — by checking `Date.now() -`

`unlockedAt < TIMEOUT_MS` (10–15 min is reasonable for this kind of app).

- Use the Page Visibility API to catch a tab coming back into focus after being hidden a while, and re-check the timeout then — that's what mimics "reopening the app" rather than "still using it."
- Clear the `localStorage` key on explicit logout or a manual lock button — that locks every open tab at once.

Multiple devices need no special handling: since the unlock state lives in each browser's local storage rather than the server, unlocking on your phone has zero effect on your laptop. Actively avoid making "vault unlocked" a server-side flag — that would make one device's unlock apply everywhere, which is worse, not better.

Multi-user isolation

Already solid: `user_id` + RLS (`auth.uid() = user_id`) is enforced by Postgres itself, on every query, regardless of the client code — a buggy fetch that forgets a filter still can't return someone else's rows. That's a stronger guarantee than an app-level `if user.id === owner.id` check, which only protects you if every code path remembers it.

Households change the policy shape slightly — a shared resource needs to pass if the requester owns it *or* belongs to the household it's attached to:

```sql
create policy "own or household data"
on accounts for select
using (
  auth.uid() = user_id
  or household_id in (
    select household_id from household_members where user_id = auth.uid()
  )
);
```

Worth deciding per table whether every member can write (add expenses) vs. only certain roles can (remove a shared account) — that's a product call, not a security one.

Design / mobile / hover issues

The single most likely cause of "broken on phone" for anything interactive: hover-dependent UI. Touchscreens have no true hover state, so if an action (edit icon, delete button) only appears via `:hover`, it's literally unreachable on mobile. Scope hover effects to devices that actually support them:

```css
@media (hover: hover) and (pointer: fine) {
  .row .action-icon { opacity: 0; }
  .row:hover .action-icon { opacity: 1; }
}
```

On a touchscreen that query is false, so you'd fall back to always-visible icons or a tap/swipe reveal instead.

For layout, the usual suspects: a missing `<meta name="viewport" content="width=device-width, initial-scale=1">`, fixed pixel widths instead of `%`/`rem`/flex-wrap, absolutely-positioned elements that don't reflow at narrow widths, and tap targets under ~44px. Can't fix specifics without seeing the actual components, though — share the CSS/component code or screenshots of the broken views for a real pass.

Bill reminders + balance sync

- Reminder: a daily scheduled job (Supabase's `pg_cron`, or a Vercel Cron job hitting an API route) checks `bills` where `next_due_date` is within N days and `is_paid = false`, fires a notification. Simplest version is an in-app banner on next visit; email (Resend, or Supabase's built-in mail) or push if you want it outside the app.
- Confirmation: "mark paid" creates a `transactions` row (expense, using the bill's `category_id`), decrements the linked account's balance, sets `is_paid = true`, and — if `frequency != 'one-time'` — computes the new `next_due_date` and resets `is_paid` for the next cycle.
- Auto-detect ("connect it with the account") has two very different scopes. Doable now: when a transaction is manually added, match it against pending bills by amount + date proximity + category, and prompt "does this cover [bill]?" Full bank-linked auto-detection is a much bigger step — in India that goes through the RBI's Account Aggregator framework rather than something like Plaid, and means becoming (or partnering as) a registered data-consuming entity operating under RBI oversight, typically through a licensed Account Aggregator partner that handles the compliance layer — realistically out of scope for a personal project. Start with the heuristic-matching version. Medium Productgrowth

Market data beyond Yahoo Finance

The free-tier landscape has shifted a lot — quite a few providers have shut down entirely or moved to paid-only models, though a handful of genuinely useful free options remain. For hobby scale: GitHub

- Alpha Vantage has a free tier covering stocks, forex, and crypto, capped at 5 requests/minute and 500/day — plenty for a dashboard that isn't polling constantly. EODHD
- Twelve Data is currently the closest like-for-like replacement for Yahoo Finance's behavior, and a common pick for hobby/prototype-scale projects. GitHub
- For crypto, CoinGecko's free "Demo" plan needs no paid signup and allows 100 calls/minute up to 10,000 calls/month — generous enough you probably wouldn't need your `market_cache` layer just for that piece. Zyla API Hub

The catch is Indian equities — none of the above cover NSE/BSE well. What actually returns that data day to day is mostly the same category your current setup is already in: free, keyless community APIs, some just reformatting Yahoo Finance with Indian ticker conventions, others scraping NSE's own public site directly — same "could break if the source changes" risk your Yahoo Finance setup already carries. For something steadier

with an actual support tier, IndianAPI.in runs free/hobby plans through paid developer and analyst tiers specifically for NSE/BSE data. Practically: keep Yahoo Finance or Twelve Data for anything global, treat Indian equities as its own smaller integration behind your existing `market_cache` layer — that's exactly what protects you if the underlying source hiccups. GitHub glama

Fuel prices by city + the city switcher

Same shape as the market-data problem: there are commercial APIs returning daily petrol, diesel, and CNG rates by state and city, and also free, open-source scrapers that expose city/district-level fuel prices as a small self-hosted API. Your schema already lists `fuel` as a valid `market_cache.type`, so this slots into the exact same caching pattern you're using for stocks — a scheduled job pulls the day's price for whichever cities your users have selected, writes it into `market_cache`, and your `/fuel` and `/market` routes both read from that table instead of hitting a live source per request. RapidAPI apple

For the switcher itself: a combobox with client-side fuzzy search (Fuse.js or `cmdk`) over a static list of Indian cities gives you prefix + typo-tolerant matching with no server round-trip — genuinely "hybrid" in the sense you mean. Animate it with Framer Motion (already in your stack) for open/close and selection so it matches the rest of the UI, debounce keystrokes by ~150ms, and persist the chosen city in `settings` so it's remembered per user instead of reset every login.

Paste the v2 digest's actual text and I can fold today's decisions (households RLS, bill-reminder schema additions) into an updated version. For a real design pass on the mobile issues, share the actual component/CSS files and I'll get specific instead of general.